

Aether Protocol: Specification v0.1

Generated Specification

April 29, 2025
Version 0.1 (Draft)

Contents

1	Introduction	3
1.1	Goal	3
1.2	Context	3
1.3	Core Principles	3
1.4	Target Audience	3
1.5	Document Status	3
2	Architecture Overview	3
3	Core Concepts & Definitions	4
4	Protocol Layers	5
4.1	BFT Consensus Layer (Foundation)	5
4.2	Commitment Layer	5
4.3	Verification Layer	5
4.4	Adjustment Layer	6
4.5	Dispute Resolution Layer	6
4.6	Enforcement Layer	7
5	Data Types and Formats (Examples)	8
6	Security & Cryptography	9
6.1	Assumptions	9
6.2	ZKP Usage (Minimal V1)	9
7	Operational Considerations	9
8	Governance (Minimal V1)	9
9	Open Questions & RFC Areas	10
10	Conclusion	10
A	Example Adjustment Rules	11
A.1	Contribution-Weighted Delta Adjustment (Quota Regulation)	11
A.1.1	Purpose	11
A.1.2	Assumptions for EFS	11
A.1.3	Parameters defined in Rule	11
A.1.4	Algorithm Steps	11
A.1.5	Example Walkthrough	12

A.2	Direct Compensation Delta Rule	12
A.2.1	Purpose	12
A.2.2	Assumptions for EFS	12
A.2.3	Parameters defined in Rule	13
A.2.4	Algorithm Steps	13

1 Introduction

1.1 Goal

To define a decentralized, automated, and cryptographically secure protocol for coordinating the management of measurable, cross-border *economic externalities* arising from environmental impacts among participating sovereign or regional agents.

1.2 Context

Existing international environmental agreements often suffer from lack of robust enforcement, reliance on trust or political goodwill, difficulty in handling adversarial behavior, and challenges in agreeing on subjective valuations. This protocol aims to address these limitations through decentralization, cryptographic verification, and automated economic incentives/disincentives.

1.3 Core Principles

- **Decentralization:** No central authority or trusted third party. Peer-to-peer network operation.
- **Byzantine Fault Tolerance (BFT):** Protocol integrity and liveness maintained assuming fewer than 1/3 of participants are malicious or faulty.
- **Verifiable Economic Externalities:** Focus *exclusively* on externalities manifesting as quantifiable, verifiable economic impacts (changes in costs or utility) on producers or consumers within participating jurisdictions.
- **Cryptographic Enforcement:** Utilizes digital signatures, cryptographic commitments, and Zero-Knowledge Proofs (primarily for fraud proofs/computational integrity) to ensure data integrity and compliance.
- **Incentive Compatibility:** Integrated economic mechanisms (collateral staking and slashing) align rational self-interest with protocol compliance and truthful reporting.
- **Delta-Only Adjustments:** Obligations (e.g., quotas, compensation) are adjusted based *only* on verifiable changes (deltas) in aggregated, measured externalities, avoiding complex baseline negotiations.
- **Simplicity (Zero-Mass Design):** Focus on the minimal core components required to achieve the value proposition. Defer non-essential features (YAGNI - You Ain't Gonna Need It).

1.4 Target Audience

Engineering teams responsible for implementing and deploying the protocol.

1.5 Document Status

Version 0.1 - Initial Draft. Certain parameters and mechanisms are marked for further definition via an RFC (Request for Comments) process involving stakeholders and technical experts.

2 Architecture Overview

The protocol operates as a replicated state machine run by participating agents (nodes). State transitions are driven by participant actions and external data, ordered and validated via BFT consensus. The logical flow follows distinct layers, as depicted conceptually in Figure 1.

Figure Placeholder: Protocol Layer Interaction Diagram

This diagram illustrates the flow between protocol layers (Commitment, Verification, Adjustment, Dispute Resolution, Enforcement) mediated by the central BFT Consensus Layer, which manages the core protocol state machine (including logs/records for commitments, verified data, adjustments, disputes, penalties, reputation, and stake). External data enters via Oracles into the Verification Layer. Agent actions initiate the process at the Commitment Layer.

Figure 1: High-level architecture of the Aether Protocol.

3 Core Concepts & Definitions

- **Agent:** A participating entity (e.g., nation, regional authority) running a protocol node, identified by a unique cryptographic key pair (**AgentID**).
- **Externality Function Specification (EFS):** A precise, agreed-upon definition module for a *specific* cross-border externality. Each EFS *must* include:
 - **EFS_ID:** Unique identifier.
 - **Description:** Clear language description of the externality (e.g., "SO₂ emissions from Zone A impacting agricultural yields in Zone B").
 - **ActionType:** The type of action by the "causer" agent being measured (e.g., SO₂ emissions rate, water withdrawal volume). It may involve setting Quotas.
 - **ImpactType:** The type of economic impact on the "affected" agent being measured (e.g., monetary loss in crop value, increased water treatment cost).
 - **AttributionModel:** A deterministic algorithm or verifiable methodology linking **ActionType** quantities to **ImpactType** quantities, potentially considering geographic/temporal factors. *Must be computable based on data provided by the Verification Layer.*
 - **VerificationDataSpec:** Specifies the required data inputs from Oracles (environmental monitoring, economic stats) needed to evaluate the **AttributionModel**.
 - **AdjustmentRule:** The formula determining how obligations change based on the calculated aggregate impact delta (see Section 4.4 and Appendix A).
- **Verifiable Economic Impact:** The quantified economic consequence (ΔCost or $\Delta\text{Utility}$ in monetary terms) calculated using an approved EFS and verified data.
- **Resource Action Commitment:** A binding cryptographic commitment made by an agent regarding its planned actions relevant to one or more EFSs for a future period.
- **Epoch:** A discrete time period (e.g., monthly, quarterly) for which commitments are made and adjustments are calculated.
- **Collateral (Stake):** Digital assets locked by an agent within the protocol's control as a guarantee against non-compliance.
- **Slashing:** The automated confiscation of an agent's collateral upon verifiable proof of protocol violation.

4 Protocol Layers

4.1 BFT Consensus Layer (Foundation)

- **Function:** Ensures all honest agents agree on the same ordered sequence of valid state transitions (commitments, verified data hashes, adjustments, penalties, etc.) despite up to $f < n/3$ Byzantine agents (where n is the total number of agents). Implements State Machine Replication.
- **Requirements:** Safety (no forks in agreed state), Liveness (valid transactions eventually committed), Byzantine Fault Tolerance.
- **Technology:** Recommend mature BFT protocols suitable for permissioned/consortium settings (e.g., CometBFT/Tendermint, HotStuff variants). Performance characteristics (latency, throughput) must be adequate for the expected number of agents and transaction volume. (See RFC-09).
- **State:** The consensus maintains the canonical state, including agent registry, active EFSs, commitment log, verified data registry (hashes/proofs), adjustment records, collateral balances, reputation scores, and dispute status.

4.2 Commitment Layer

- **Function:** Allows agents to make binding, non-repudiable commitments about future actions relevant to active EFSs.
- **Process:**
 1. Agent i determines its planned **ResourceAction** $a_{i,t+1}$ for the next **Epoch** $t + 1$.
 2. Agent i generates a secret random nonce r_i .
 3. Agent i computes **Commitment** $C_{i,t+1} = \text{Hash}(a_{i,t+1} || r_i || \text{Epoch}_{t+1})$. The $||$ operator denotes concatenation.
 4. Agent i signs $C_{i,t+1}$ and broadcasts it as a transaction to the BFT Consensus Layer before the deadline for **Epoch** $t + 1$.
- **Cryptography:** Standard collision-resistant hash function (e.g., SHA-256), standard digital signatures (e.g., ECDSA).

4.3 Verification Layer

- **Function:** Securely imports external data required by active EFSs, verifies agent actions against commitments, calculates attributable impacts according to EFSs, and submits results to the BFT layer. Handles inherent data lags.
- **Components:**
 - **Oracles:** Interface with external data sources. MUST use decentralized oracle networks (DONs) or sources providing strong cryptographic attestations of data integrity (e.g., signed data feeds from trusted institutions). Oracles fetch data specified in **VerificationDataSpec** for relevant EFSs (e.g., sensor readings, satellite analysis results, official economic stats). (See RFC-02).
 - **Verification Nodes/Logic:** May be run by all agents or a designated (potentially rotating or specialized) subset.
- **Process** (for Epoch t , occurs during Epoch $t + k$ where $k \geq 1$ is verification lag):

1. Oracles fetch required environmental and economic data pertaining to Epoch t . Data is cryptographically signed/attested by the source/DON.
 2. Verification logic retrieves Agent i 's commitment $C_{i,t}$. Agent i (or the verification logic, if actions are publicly observable) provides the revealed action $a_{i,t}$ and nonce r_i . Verification: check $\text{Hash}(a_{i,t}||r_i||\text{Epoch}_t) == C_{i,t}$. *Fraud Proof*: If $a_{i,t}$ derived from oracle data differs significantly from the revealed commitment, a fraud proof can be generated.
 3. For each active EFS involving agent i as a potential causer and agent j as potentially affected: Use the fetched oracle data and the verified action $a_{i,t}$ as input to the EFS **AttributionModel** to calculate the Verifiable Economic Impact $V_{i \rightarrow j,t}$.
 4. (Optional ZKP): If the **AttributionModel** involves sensitive inputs or complex computations, Agent i (or the computation provider) can generate a ZKP π proving that $V_{i \rightarrow j,t}$ was computed correctly according to the EFS specification from committed/verified inputs, without revealing all inputs. Verification logic checks proof π .
 5. Submit results (verified action $a_{i,t}$, calculated impacts $V_{i \rightarrow j,t}$, relevant data hashes, proofs π if any) as transactions to the BFT Consensus Layer. Store raw/detailed data off-chain (e.g., IPFS, Agent-hosted) referenced by hash.
- **Lag Handling**: The protocol explicitly operates with a lag k . Adjustments for Epoch t are calculated and applied only after verification completes in Epoch $t + k$.

4.4 Adjustment Layer

- **Function**: Calculates and records adjustments to agent obligations based *only* on the *delta* of verified aggregate economic impacts.
- **Process** (Runs periodically, e.g., at end of Epoch $t + k$ using verified data from Epoch t):
 1. For each EFS, aggregate the verified economic impacts across all affected agents: $V_{\text{total},t} = \sum_j \sum_{i \neq j} V_{i \rightarrow j,t}$. Aggregation occurs via BFT consensus (e.g., reading values from the state).
 2. Calculate the delta: $\Delta_t = V_{\text{total},t} - V_{\text{total},t-1}$ (or compared to a fixed baseline V_{baseline} for the first few epochs).
 3. Apply the **AdjustmentRule** defined in the EFS (or a global default). This rule takes Δ_t and potentially the attributable verified impacts $V_{i \rightarrow j,t}$ as input. (See Appendix A for examples and RFC-06).
 4. The rule outputs specific **Obligations** for agents (e.g., "Agent i owes compensation X ", "Agent i 's quota for **ActionType** is reduced by $Y\%$ ").
 5. Record these **Obligations** in the BFT state for Epoch t . Note: The protocol records the obligation; enforcement (e.g., verification of actual compensation payment if required by the rule) relies on the Enforcement Layer and potentially Oracles.

4.5 Dispute Resolution Layer

- **Function**: Provides a mechanism for agents to challenge specific protocol operations or data based on cryptographic evidence. *Scope (V1)*: Focus on disputes resolvable via protocol rules and cryptographic proofs.
- **Triggers**:

- Challenge to Oracle data validity (requires conflicting signed data from another trusted source or proof of oracle malfunction).
 - Challenge to commitment opening (Agent i reveals a', r' not matching C_i).
 - Challenge to EFS calculation integrity (requires ZKP failure if ZKPs are used, or demonstrable inconsistency with EFS logic based on public data).
 - Challenge to Adjustment calculation (demonstrable error based on BFT state data and Adjustment Rule).
- **Process:**
 1. Challenger submits a signed dispute transaction detailing the claim and evidence (e.g., conflicting data hash, failed proof verification).
 2. Accused party (Agent or Oracle) responds with counter-evidence (e.g., revealing nonce, providing valid proof).
 3. Resolution: Determined automatically by protocol logic based on cryptographic verification and pre-defined rules. If evidence confirms violation (e.g., commitment mismatch, failed ZKP verification, invalid data signature), the challenge succeeds. Otherwise, it fails.
 - **RFC-05:** Mechanisms for resolving disputes requiring subjective judgment (e.g., fundamental challenges to EFS `AttributionModel` accuracy) are out of scope for V1 and require a separate governance/expert panel process defined via RFC. Handling the "honest mistake" defense against slashing falls under this layer's procedures (requiring evidence of non-malicious error).

4.6 Enforcement Layer

- **Function:** Enforces protocol rules and obligations through automated economic consequences.
- **Components:**
 - **Staking:** Agents must deposit and maintain a minimum collateral amount S (denominated in a pre-approved stablecoin or digital asset) into a protocol-controlled smart contract. S is TBD (RFC-03). Failure to maintain stake may result in suspension.
 - **Slashing:** Automatic confiscation of a portion (or all) of staked collateral. Triggered by:
 - * Losing a dispute in the Dispute Resolution Layer (e.g., proven false commitment, submitted invalid data).
 - * Failure to meet an Obligation recorded by the Adjustment Layer by its deadline (e.g., non-payment of required compensation, requires Oracle confirmation if payment is external). Slashing percentage/amount is TBD (RFC-04).
 - **Reputation (Minimal V1):** Each agent has a numerical reputation score stored in the BFT state. Initialized at a default value. Decreases upon slashing events. Increases slowly over time with consistent compliance. *V1 Usage:* Primarily informational. (See RFC-07).
- **Slashing Destination:** Slashed funds TBD (RFC-04 - e.g., burned, distributed to affected parties if applicable, sent to protocol treasury controlled by governance).

5 Data Types and Formats (Examples)

Example data structures (conceptual; final formats TBD, e.g., Protocol Buffers):

```
1 AgentID: PublicKey / Address # e.g., 20-byte Ethereum-style address
2 Timestamp: Int # Unix timestamp (seconds)
3 EpochID: Uint # Sequential integer
4 Hash: Bytes # 32-byte hash (e.g., SHA-256)
5 Signature: Bytes # Standard cryptographic signature (e.g., ECDSA)
6 Amount: String # Fixed-point decimal number for monetary values
7 Percentage: String # Fixed-point decimal number for percentages
8 EFS_ID: String # UUID or unique identifier string
9 AssetType: String # Identifier for collateral asset type
10
11 Commitment: {
12     AgentID: AgentID,
13     EpochID: EpochID,
14     CommitmentHash: Hash,
15     Signature: Signature
16 }
17
18 VerifiedImpact: {
19     EFS_ID: EFS_ID,
20     CauserAgentID: AgentID,
21     AffectedAgentID: AgentID,
22     EpochID: EpochID,
23     ImpactAmount: Amount,
24     DataSourceHash: Hash,
25     Proof: Optional<Bytes>, # e.g., ZKP
26     ReporterAgentID: AgentID,
27     Signature: Signature
28 }
29
30 Obligation: {
31     EFS_ID: EFS_ID,
32     ObligatedAgentID: AgentID,
33     EpochID: EpochID, # Epoch the impact relates to
34     ObligationType: Enum('Compensate', 'QuotaAdjust'),
35     Value: Amount | Percentage, # Depends on ObligationType
36     BeneficiaryAgentID: Optional<AgentID> # For compensation
37 }
38
39 StakeRecord: {
40     AgentID: AgentID,
41     CollateralAmount: Amount,
42     AssetType: AssetType
43 }
44
45 SlashingEvent: {
46     AgentID: AgentID,
47     EpochID: EpochID, # Epoch the slash occurred
48     AmountSlashed: Amount,
49     ReasonCode: Enum, # e.g., DisputeLost, NonCompliance
50     DisputeID: Optional<Uint> # Link to dispute if applicable
51 }
52
53 ReputationScore: {
54     AgentID: AgentID,
55     ScoreValue: Int,
```



```

56 | LastUpdateEpoch: EpochID
57 | }

```

Listing 1: Example Data Types

Note: Final serialization formats need careful definition for interoperability.

6 Security & Cryptography

6.1 Assumptions

- **BFT Consensus:** Secure if fewer than 1/3 of participants are Byzantine.
- **Cryptography:** Standard hardness assumptions for chosen hash functions (collision resistance), digital signatures (unforgeability), and ZKPs (soundness, zero-knowledge - note: soundness against single honest prover/verifier finding fraud is often sufficient for practical purposes).
- **Oracle Security:** Trust in the chosen DON's crypto-economic security or the attestation mechanism of trusted institutional sources. Data *source* integrity is an external dependency.
- **Secure Implementation:** Protocol software is implemented correctly without exploitable bugs.

6.2 ZKP Usage (Minimal V1)

- **Primary:** Fraud proofs (proving inconsistency between committed and actual/verified data).
- **Secondary (Optional):** Proving correct execution of EFS `AttributionModel` or Adjustment Rules if computed privately/off-chain.
- Focus on efficiency and availability of robust libraries (e.g., SNARKs like Groth16/Plonk or STARKs depending on setup requirements and performance trade-offs).

7 Operational Considerations

- **Bootstrapping:** Requires a Genesis state defining initial participating Agents, their public keys, initial stake deposits, approved EFS specifications, initial protocol parameters (epoch duration, stake requirements, slashing rules - subject to RFC), and the BFT validator set.
- **Participant Onboarding:** New agents require approval via the governance mechanism (see Section 8) and must deposit the required collateral.
- **EFS Management:** Adding or modifying EFSs requires approval via the governance mechanism. EFS specifications must be unambiguous and implementable. (See RFC-01).

8 Governance (Minimal V1)

- **Initial Parameters:** Set in the Genesis state.

- **Updates:** Changes to protocol parameters, approved EFSs, or core protocol logic require a formal proposal submitted as a transaction and approved by BFT consensus vote, requiring a supermajority (e.g., $> 2/3$) of participating agents (stake-weighted voting TBD - See RFC-08).
- **RFC-08:** Development of more sophisticated governance mechanisms (e.g., dedicated governance council, liquid democracy, integration with reputation) is deferred.

9 Open Questions & RFC Areas

The following require further specification through a collaborative RFC process:

- **RFC-01:** Specific EFS Definitions: Detailed specification for initial 1-3 high-priority externalities.
- **RFC-02:** Oracle Integration Details: Choice of specific DONs/attested sources, data formats, verification procedures, handling oracle disputes.
- **RFC-03:** Staking Parameters: Minimum collateral amount S , approved asset types, mechanisms for adjusting S .
- **RFC-04:** Slashing Rules: Specific percentages/amounts for different violation types, destination of slashed funds.
- **RFC-05:** Dispute Resolution for Subjective Claims: Process for handling challenges to EFS model validity or situations requiring expert judgment (potential expert panels, appeals process). Defining "honest mistake" criteria and procedures.
- **RFC-06:** Adjustment Rules: Specification of additional adjustment rule types beyond examples in Appendix A.
- **RFC-07:** Reputation System Mechanics: Concrete impacts of reputation scores (if any beyond informational in V1).
- **RFC-08:** Governance Evolution: More advanced governance models beyond simple voting.
- **RFC-09:** BFT Performance Scalability: Detailed analysis for expected network size and transaction load.
- **RFC-10:** Legal Political Integration: Strategy for compatibility with existing frameworks.
- **RFC-11:** Handling Non-Quantifiable Harms: Mechanisms or policy interfaces to address environmental impacts outside the scope of verifiable economic externalities.

10 Conclusion

The Aether Protocol specification (v0.1) provides a blueprint for a minimal, robust, and decentralized system focused on managing verifiable economic externalities across borders. By leveraging BFT consensus, cryptography, and integrated economic enforcement, it aims to create a predictable and incentive-compatible environment for cooperation among potentially adversarial agents. Successful implementation relies on careful selection of technologies, precise definition of EFSs, and collaborative refinement of parameters and mechanisms through the specified RFC process.

A Example Adjustment Rules

These examples illustrate potential `AdjustmentRule` logic that could be defined within an EFS. The specific rule applied depends on the EFS governing a particular interaction.

A.1 Contribution-Weighted Delta Adjustment (Quota Regulation)

A.1.1 Purpose

Adjusts emission/action quotas based on the change (delta) in total verified economic impact, weighted by each agent's contribution to that impact.

A.1.2 Assumptions for EFS

- Externality involves a controllable `ActionType` (e.g., SO₂ emissions) with assigned quotas Q_i .
- Goal is to stabilize or reduce total aggregate economic impact V_{total} .
- Verification lag k exists. Adjustments decided at end of Epoch $t + k$ apply to quotas for Epoch $t + k + 1$.

A.1.3 Parameters defined in Rule

- `BaselineImpact` (V_{baseline}): The initial target or average impact level.
- `SensitivityFactor` (α): Positive dimensionless factor (e.g., 1.0).
- `MinimumQuota` (Q_{min}): Absolute minimum allowed quota.
- `MaximumQuota` (Q_{max}): Absolute maximum allowed quota.

A.1.4 Algorithm Steps

(Executed by Adjustment Layer at end of Epoch $t + k$)

1. **Retrieve Verified Data:** From BFT state for Epoch t : Total impact $V_{\text{total},t}$, impact caused by agent i $V_{i \rightarrow \text{total},t}$, previous total impact $V_{\text{total},t-1}$ (or V_{baseline} if $t = 0$), and current quota for agent i $Q_{i,t+k}$.

2. **Calculate Aggregate Impact Delta:**

$$\Delta_t = V_{\text{total},t} - V_{\text{total},t-1} \quad (\text{or } V_{\text{total},t} - V_{\text{baseline}} \text{ if } t = 0)$$

3. **Determine Adjustment Trigger Preliminary New Quota:**

- If $\Delta_t \leq 0$: Set preliminary new quota $Q'_{i,t+k+1} = Q_{i,t+k}$ (V1: No automatic loosening).
- If $\Delta_t > 0$: (Quota tightening required)
 - (a) Calculate contribution share (if $V_{\text{total},t} > 0$, else 0):

$$s_{i,t} = V_{i \rightarrow \text{total},t} / V_{\text{total},t}$$

- (b) Calculate relative impact increase (handle division by zero if denominator is 0):

$$r_t = \Delta_t / V_{\text{total},t-1} \quad (\text{or } \Delta_t / V_{\text{baseline}} \text{ if } t = 0)$$

(c) Calculate reduction factor for agent i :

$$f_{i,t} = \alpha \times r_t \times s_{i,t}$$

(d) Calculate preliminary new quota:

$$Q'_{i,t+k+1} = Q_{i,t+k} \times \max(0, 1 - f_{i,t})$$

4. Apply Floor and Ceiling:

$$Q_{i,t+k+1} = \max(Q_{\min}, \min(Q_{\max}, Q'_{i,t+k+1}))$$

5. **Record Obligation:** Record $Q_{i,t+k+1}$ in BFT state as the **Obligation** (active quota) for Agent i for Epoch $t + k + 1$.

A.1.5 Example Walkthrough

Assume Epoch $t = 5$, lag $k = 1$. Deciding quotas for Epoch 7 at end of Epoch 6. Agents A, B. Quotas Q_A, Q_B . Parameters: $\alpha = 1.0$, $Q_{\min} = 10$, $Q_{\max} = 200$. Data for Epoch 5: $V_{A \rightarrow \text{total},5} = \1000 , $V_{B \rightarrow \text{total},5} = \$500 \implies V_{\text{total},5} = \1500 . Data for Epoch 4: $V_{\text{total},4} = \$1200$. Current Quotas (for Epoch 6): $Q_{A,6} = 100$ T/day, $Q_{B,6} = 80$ T/day.

- Step 2: $\Delta_5 = \$1500 - \$1200 = \$300$.
- Step 3: $\Delta_5 > 0$, tightening needed.
- Step 3a (Contribution Share): $s_{A,5} = 1000/1500 = 2/3$. $s_{B,5} = 500/1500 = 1/3$.
- Step 3b (Relative Increase): $r_5 = 300/1200 = 0.25$.
- Step 3c (Reduction Factor): $f_{A,5} = 1.0 \times 0.25 \times (2/3) \approx 0.167$. $f_{B,5} = 1.0 \times 0.25 \times (1/3) \approx 0.083$.
- Step 3d (Preliminary New Quota): $Q'_{A,7} = 100 \times (1 - 0.167) = 83.3$. $Q'_{B,7} = 80 \times (1 - 0.083) \approx 73.36$.
- Step 4 (Apply Floor/Ceiling): $Q_{A,7} = 83.3$. $Q_{B,7} = 73.36$.
- Step 5 (Record): Obligation recorded: New quota for A is 83.3, for B is 73.36 for Epoch 7.

A.2 Direct Compensation Delta Rule

A.2.1 Purpose

Requires agents causing an increase in aggregate economic impact to financially compensate affected agents, proportional to their contribution to the increase.

A.2.2 Assumptions for EFS

- EFS `AttributionModel` can link causer i to affected j with impact $V_{i \rightarrow j,t}$.
- Goal is to compensate for *increases* in harm (V_{total}).
- Verification lag k exists.

A.2.3 Parameters defined in Rule

- **BaselineImpact** (V_{baseline}): Initial target impact level.
- **CompensationMultiplier** (γ): Factor applied to impact delta (e.g., $\gamma = 1.0$).

A.2.4 Algorithm Steps

(Executed by Adjustment Layer at end of Epoch $t + k$)

1. **Retrieve Verified Data:** From BFT state for Epoch t : $V_{\text{total},t}$, $V_{i \rightarrow \text{total},t}$, $V_{i \rightarrow j,t}$, $V_{\text{total},t-1}$ (or V_{baseline}).

2. **Calculate Aggregate Impact Delta:**

$$\Delta_t = V_{\text{total},t} - V_{\text{total},t-1} \quad (\text{or } V_{\text{total},t} - V_{\text{baseline}} \text{ if } t = 0)$$

3. **Determine Obligation Trigger:**

- If $\Delta_t \leq 0$: No compensation obligation generated. End.
- If $\Delta_t > 0$: (Compensation required) Proceed to step 4.

4. **Calculate and Allocate Compensation** (for each causer agent i):

- (a) Calculate contribution share (if $V_{\text{total},t} > 0$, else 0):

$$s_{i,t} = V_{i \rightarrow \text{total},t} / V_{\text{total},t}$$

- (b) Calculate allocated delta for agent i :

$$\Delta_{i,t}^{\text{allocated}} = \Delta_t \times s_{i,t}$$

- (c) Calculate total compensation required from agent i :

$$C_{i,t} = \gamma \times \Delta_{i,t}^{\text{allocated}}$$

- (d) Distribute compensation to affected agents j (for each j where $V_{i \rightarrow j,t} > 0$, handle division by zero if $V_{i \rightarrow \text{total},t} = 0$):

$$C_{i \rightarrow j,t} = C_{i,t} \times (V_{i \rightarrow j,t} / V_{i \rightarrow \text{total},t})$$

- (e) **Record Obligation:** Record **Obligation** in BFT state: "Agent i owes compensation $C_{i \rightarrow j,t}$ to Agent j for Epoch t ". Specify payment deadline (e.g., end of Epoch $t+k+1$).

Note: The Enforcement Layer subsequently checks (via Oracle input or bilateral confirmation) if these compensation payments are made externally by the deadline. Failure to pay triggers slashing according to RFC-04 rules.